

```
import torch

if torch.cuda.is_available():
    print("Successfully enabled CUDA processing")
    device = torch.device("cuda")
else:
    print("CUDA processing not available. Things will be slow :-(")
    device = torch.device("cpu")
```

Successfully enabled CUDA processing

Dataset

In this assignment, the dataset consists of EEG signals stored in `.npz` files. Each file corresponds to one subject's data. In particular, loading a subject's file provides two essential `numpy` arrays that we use. These two arrays can be accessed using the keys `x` and `y` respectively in the loaded `.npz` file.

- `x` (Input): The EEG signals, where each sample is a 30-second window containing 3000 sequential points across a single channel. The inputs are $X^{[n]} \in \mathbb{R}^{3000 \times 1}$.
- `y` (Target): The corresponding sleep stage labels, where each label $\tau^{[n]} \in \{0, 1, 2, 3, 4\}$ represents one of the five sleep stages: Wake, N1, N2, N3, and REM.

Please use the cell below to download and extract the data into the specified `data_directory`.

```
import pathlib
import urllib
import zipfile

# set directory where to download data to
data_directory = pathlib.Path("data")
data_file = data_directory/"Sleep-EDF-Subjects.zip"

data_url = "https://seafile.ifi.uzh.ch/f/08eda6b5e19a4b39b922/?dl=1"

# create data directory
data_directory.mkdir(exist_ok=True)

# Skip downloading if the file already exists
if not data_file.is_file():
    # Download the file
    urllib.request.urlretrieve(data_url, data_file)
    print(f"Downloaded {data_file} successfully.")

    # Extract the zip file
    with zipfile.ZipFile(data_file, 'r') as zip_ref:
        zip_ref.extractall(path=data_directory)
    print(f"Extracted {data_file} successfully.")
```

For this assignment, specific lists of subject files are provided below to define the training, validation, and test sets. **Please run the following cell to define them, and make sure to use these splits to ensure consistency across all experiments.**

```
# Define our training/validation/test splits, which we perform per subject
training_subjects = ["SC4001E0.npz", "SC4002E0.npz", "SC4011E0.npz", "SC4012E0.npz", "SC4021E0.npz", "SC4022E0.npz", "SC4031E0.npz", "SC4032E0.npz", "SC4041E0.npz", "SC4042E0.npz"]
validation_subjects = ["SC4051E0.npz", "SC4052E0.npz", "SC4061E0.npz"]
test_subjects = ["SC4062E0.npz", "SC4071E0.npz", "SC4072E0.npz"]

stage_names = {
    0: "Wake",
    1: "Light sleep",
    2: "Stable sleep",
    3: "Deep sleep",
    4: "REM"
}

O = len(stage_names)
```

Task 2.1: Dataset Loading

Implement a function that takes a list of subject file names and loads the corresponding data to create the full input X and target τ (`torch.Tensor`) for the specified set (training, validation, or test). **The data from all subject files of a set should be concatenated.** Ensure that:

- The input X tensor is of type `torch.float32` and has the shape $N \times C \times S$ (**Number of samples, Channels, Sequence length**). *This format is necessary because the feature extractor expects the input in this order.*
- The target T tensor is of type `torch.long` and has the shape N (**Number of samples,**).

```
import numpy

def load_data(subject_list):
    """
    Load and concatenate EEG sleep data from multiple .npz files.

    Parameters
    -----
    subject_list : list of str
        Subject file names containing data['x'] (EEG signals) and data['y'] (labels).

    Returns
    -----
    X : torch.FloatTensor (N, 1, 3000)
    T : torch.LongTensor (N,)
    """

    # Read all subject files of a set
    inputs, targets = [], []

    for subject in subject_list:
        data = numpy.load(data_directory/subject)
        inputs.append(data['x'])
        targets.append(data['y'])

    X_np = numpy.vstack(inputs)
    T_np = numpy.concatenate(targets)

    X = torch.from_numpy(X_np).float().view(-1, 1, 3000)
    T = torch.from_numpy(T_np).long()

    return X, T

# We load the three different sets
X_train, T_train = load_data(training_subjects)
X_valid, T_valid = load_data(validation_subjects)
X_test, T_test = load_data(test_subjects)
```

✓ Test 1: Data Types and Sizes

Verify that all tensors have the correct shape and data type.

```
# Define datasets and expected shapes
sets = {
    "train": (X_train, T_train, 10589),
    "valid": (X_valid, T_valid, 2761),
    "test": (X_test, T_test, 3265)
}

for set, (X, T, expected_length) in sets.items():
    assert isinstance(X, torch.Tensor) and isinstance(T, torch.Tensor), f"{set}: X or T is not a torch.Tensor"
    assert X.dtype == torch.float32, f"X_{set} should be float32"
    assert T.dtype == torch.long, f"T_{set} should be long"
    assert X.shape == (expected_length, 1, 3000), f"X_{set} shape should be ({expected_length}, 1, 3000)"
    assert T.shape == (expected_length,), f"T_{set} shape should be ({expected_length},)"

print("Sanity check passed! All datasets are correctly loaded.")
```

Sanity check passed! All datasets are correctly loaded.

✓ Task 2.2: Data Visualization

In this task, we will explore the dataset by creating two different visualizations to better understand its structure.

First, randomly select one EEG segment for each different sleep stage label (0 to 4) from **the training set** and plot them. On the x-axis, display the corresponding sleep stage names (e.g., Wake, Light Sleep, etc.) centered under each segment. The y-axis should represent the EEG signal amplitude.

As usual, try to avoid using `numpy` functionality and rely solely on `torch` and `matplotlib` functions.

```
from matplotlib import pyplot

torch.manual_seed(42)
```

```
# ----- EEG signal segments plot -----

# Colors for each sleep stage
stage_colors = {0: "#1f77b4", 1: "#ff7f0e", 2: "#2ca02c", 3: "#d62728", 4: "#9467bd"}

pyplot.figure(figsize=(14, 4))
start = 0
segment_length = 3000 # Each EEG segment has 3000 samples

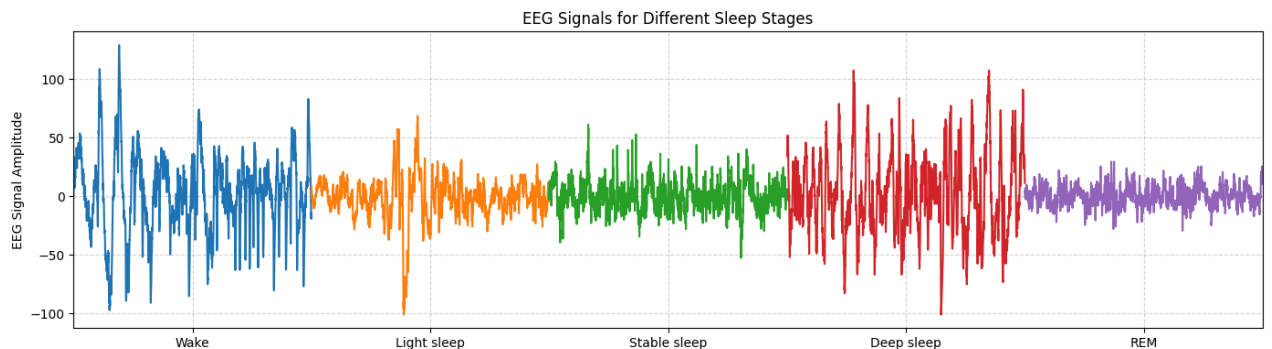
for label in range(0):

    # Find all indices where the label matches the current sleep stage in the training set
    indices = torch.where(T_train == label)[0]
    # Randomly select one index from indices
    random_idx = indices[torch.randint(0, len(indices), (1,))].item()
    # Select the EEG signals of that index and flatten to (3000,)
    eeg_signals = X_train[random_idx].flatten()

    # Plot the EEG signal with correct offset on the x-axis
    end = start + len(eeg_signals)
    pyplot.plot(range(start, end), eeg_signals, color=stage_colors[label])
    start = end # Update start for the next segment

# Set x-axis ticks to stage names centered under each segment
tick_positions = [segment_length * o + segment_length // 2 for o in range(0)]
tick_labels = [stage_names[o] for o in range(0)]
pyplot.xticks(tick_positions, tick_labels)
pyplot.xlim(0, end)

# Label the y-axis as EEG amplitude
pyplot.ylabel("EEG Signal Amplitude")
pyplot.title("EEG Signals for Different Sleep Stages")
pyplot.grid(True, linestyle="--", alpha=0.5) # Add light grid
pyplot.tight_layout()
pyplot.show()
```



Second, plot **the class distribution** for the **training**, **validation**, and **test** sets. Create three bar plots (one for each set) displayed side-by-side. Each bar should show the number of samples for each sleep stage (Wake, Light Sleep, Stable Sleep, Deep Sleep, REM).

These visualizations will help you better understand the structure of the data and the level of class imbalance across the different dataset splits.

Hint: The function `torch.unique` might be of help for counting occurrences of labels.

```
# ----- Class distribution plots -----

# Create subplots
fig, axs = pyplot.subplots(1, 3, figsize=(18, 5))

# Loop over each dataset split
for ax, set_name, labels in zip(axs, ("Training", "Validation", "Test"), (T_train, T_valid, T_test)):

    # Count occurrences of each label
    unique_labels, per_class_counts = torch.unique(labels, return_counts=True)

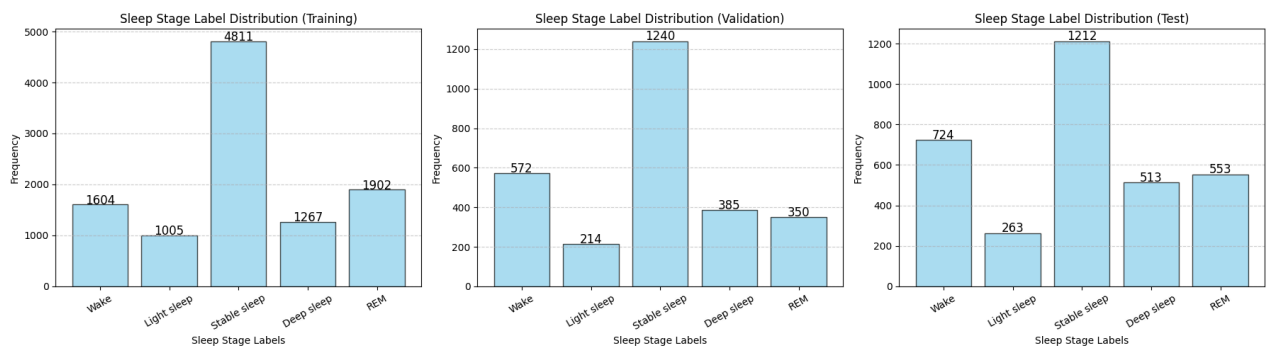
    # Plot the distribution as a bar chart
    ax.bar(unique_labels.tolist(), per_class_counts.tolist(), color='skyblue', edgecolor='black', alpha=0.7)

    # Annotate each bar with the exact count
    for i, count in enumerate(per_class_counts):
        ax.text(unique_labels[i].item(), count.item() + 0, str(count.item()), ha='center', fontsize=12)

# Set titles and axis labels
```

```
ax.set_title(f"Sleep Stage Label Distribution ({set_name})")
ax.set_xlabel("Sleep Stage Labels")
ax.set_ylabel("Frequency")
ax.set_xticks(range(0))
ax.set_xticklabels(stage_names.values(), rotation=30)
ax.grid(axis='y', linestyle="--", alpha=0.6)
```

```
pyplot.tight_layout()
pyplot.show()
```



Task 2.3: Class Weights

In later parts of the code, we will make use of class weighting strategies in order to automatically balance the loss function during training. This requires to obtain class weights, which can be computed using the equation introduced in Lecture 7:

$$\lambda_{\tau} = \frac{N}{O \cdot N_{\tau}}$$

Here, N defines the total number of samples in our dataset, while N_{τ} is the number of samples of class τ , and O is the number of classes. We can also dynamically compute these weight per batch, to arrive at the **DynaMit** weighting.

Define a function to compute the class weights from a given set of target indexes $\vec{\tau} \in \{1, \dots, O\}^N$, which can either be the **global** set of targets, or a batch of targets.

Note: When we provide a batch of targets, the number of samples for one class N_{τ} can be zero. You need to decide how you handle this.

```
def class_weights(targets):
    """
    Compute class weights to handle imbalance.

    Parameters
    -----
    targets : torch.Tensor
        Class labels of shape (N,)

    Returns
    -----
    weights : torch.Tensor
        Class weights for each class
    """

    number_of_samples = targets.shape[0]

    number_of_samples_per_class = torch.bincount(targets, minlength=0).float()

    weights = number_of_samples / (0 * (number_of_samples_per_class + 1e-6))

    return weights
```

Test 2: Class Weights

Here, we test the functionality of the class weights. Please try to pass this test before going on.

```
# create a list of targets, which does not include label 2
targets_ = torch.tensor([0]*40 + [1]*20 + [3]*30 + [4]*10, dtype=torch.long)

# compute class weights
weights_ = class_weights(targets_)

# assert that the values are correct; we do not test label 2
```

```
assert abs(weights_[0] - 0.5) < 1e-6
assert abs(weights_[1] - 1.) < 1e-6
assert abs(weights_[3] - 2./3.) < 1e-6
assert abs(weights_[4] - 2.) < 1e-6
```

Task 2.4: Input Data Normalization

Since the data is in different input regimes, we want to normalize the data.

Implement a function that normalizes input data of all sets using the whitening method with given mean and standard deviation.

$$X_{norm} = \frac{X - \mu}{\sigma}$$

Compute the mean μ and the standard deviation σ for your dataset.

Finally, normalize your input data using the implemented function.

Note: You can use `torch.mean()` and `torch.std()`, or apply the `sklearn.preprocessing.StandardScaler`.

```
def normalize(X, mu, sigma):
    return (X - mu) / (sigma + 1e-6)

# Compute mean and std from the training set
mu = torch.mean(X_train)
sigma = torch.std(X_train)

# Normalize all sets
X_train_norm = normalize(X_train, mu, sigma)
X_valid_norm = normalize(X_valid, mu, sigma)
X_test_norm = normalize(X_test, mu, sigma)
```

Task 2.5: Dataset and Data Loaders

We now have tensors that store data samples. These need to be transformed into a `torch.utils.data.Dataset`, which can be used with a standard `torch.utils.data.DataLoader` to create batches of data. For such simple data structures, `torch` provides such a dataset implementation in `torch.utils.data.TensorDataset`.

Create three `torch` datasets and their corresponding data loaders for each set by using `torch.utils.data.TensorDataset` and `torch.utils.data.DataLoader`. The training batch size should be large enough, e.g. $B = 256$, for the validation and test sets, you can choose any batch size of your choice.

```
from torch.utils.data import TensorDataset, DataLoader

# Define batch size
B = 256

# Dataset and Dataloader for training set
train_dataset = TensorDataset(X_train_norm, T_train)
train_dataloader = DataLoader(train_dataset, batch_size=B, shuffle=True)

# Dataset and Dataloader for validation set
valid_dataset = TensorDataset(X_valid_norm, T_valid)
valid_dataloader = DataLoader(valid_dataset, batch_size=512, shuffle=False)

# Dataset and Dataloader for test set
test_dataset = TensorDataset(X_test_norm, T_test)
test_dataloader = DataLoader(test_dataset, batch_size=512, shuffle=False)
```

Network Implementation

In this part of the assignment, your goal is to build a deep learning model that can predict sleep stages from sequential EEG signals.

To handle the long input sequences efficiently (3000 time steps), you will first implement the 1D convolutional feature extractor that you have designed in Task 2.1. This network compresses the raw EEG signal along the time dimension into several output channels Q_4 , while preserving key temporal patterns. It acts as a shared backbone for all subsequent model variants.

Once the features are extracted, you will explore four different network architectures, each modeling the temporal dependencies in a different way.

Task 2.6: 1D Convolutional Network (Feature Extractor)

In this task, you will implement the 1D convolutional feature extractor. For simplicity, you can use `torch.nn.Sequential`. This feature extractor will be used to **process raw EEG signals** and **compress the time dimension** by producing several output channels, which will later be passed to downstream modules such as RNN, LSTM, and Attention-based models, and the classifier.

Your feature extractor must follow these conditions:

1. It should consist of **four** `torch.nn.Conv1d` layers with a given number of $Q_4 = 128$ output channels.
2. You are encouraged to apply additional functions in these 4 layers such as:
 - Activation functions (e.g., ReLU, GELU) for non-linearity
 - `torch.nn.MaxPool1d` to reduce temporal resolution
 - `torch.nn.BatchNorm1d` for normalization
 - `torch.nn.Dropout` to improve generalization

```
def feature_extractor(output_channels = 128):
    """
    1D CNN feature extractor for EEG sleep staging.

    Returns
    -----
    torch.nn.Sequential
        Convolutional feature extractor producing (B, output_channels, L) features.
    """
    return torch.nn.Sequential(
        torch.nn.Conv1d(1, 32, kernel_size=7, stride=1, padding=3),
        torch.nn.BatchNorm1d(32),
        torch.nn.GELU(),
        torch.nn.MaxPool1d(kernel_size=4, stride=4),
        torch.nn.Dropout(0.1),

        torch.nn.Conv1d(32, 64, kernel_size=5, stride=1, padding=2),
        torch.nn.BatchNorm1d(64),
        torch.nn.GELU(),
        torch.nn.MaxPool1d(kernel_size=4, stride=4),
        torch.nn.Dropout(0.1),

        torch.nn.Conv1d(64, 128, kernel_size=3, stride=1, padding=1),
        torch.nn.BatchNorm1d(128),
        torch.nn.GELU(),
        torch.nn.MaxPool1d(kernel_size=2, stride=2),
        torch.nn.Dropout(0.1),

        torch.nn.Conv1d(128, output_channels, kernel_size=3, stride=1, padding=1),
        torch.nn.BatchNorm1d(output_channels),
        torch.nn.GELU(),
        torch.nn.MaxPool1d(kernel_size=2, stride=2),
        torch.nn.Dropout(0.1)
    )
```

✓ Task 2.7: Design General Sleep Stage Classifier

In this task, you will implement a **general sleep stage classification network** using PyTorch's `torch.nn.Module`. This network will support **four different architectural variants**, each built on top of **the same feature extractor** you implemented in **Task 2.6**. All these four different architectural variants make use of **the same classifier**, which is, for example, composed of the following layers:

Classifier:

- Linear layer with 64 hidden units
- GELU activation function
- Dropout with the probability of 0.3
- Linear layer with O output neurons

Model Variants:

Your task is to implement the following model variants in this general sleep stage classification network:

1. **Pooling:** *feature extractor* $\rightarrow$ *Global pooling* $\rightarrow$ *Classifier*

The goal of this model variant is to evaluate how effective the convolution-based feature extractor alone is, without relying on any additional recurrent or attention mechanisms to capture sequence dependencies. After passing the input through **the feature extractor**, you will directly apply **global pooling** across **the time/sequence dimension** of the output feature map. This global pooling operation condenses each sequence into a **single fixed-size feature vector** for each sample, **summarizing** the learned temporal features into a **compact representation**.

- In PyTorch, you can achieve this by taking the mean/max over the sequence dimension after the feature extractor.
- Feed the summarized representation into the classifier.

2. **RNN:** *feature extractor* $\rightarrow$ *RNN layer* $\rightarrow$ *Final hidden state* $\rightarrow$ *Classifier*

After the **feature extractor**, you will use a **Recurrent Neural Network (RNN)** layer to model the **temporal dependencies** over the **compressed time features** produced by the feature extractor network.

- Use the PyTorch module `torch.nn.RNN` to implement this layer.
- The RNN should consist of a **single layer** with 128 hidden units
- Be careful about setting `batch_first` parameter when creating the RNN. **The shape of the input to the RNN must align with this parameter.**
- After the RNN processes the sequence in `forward`, you must correctly determine the output that will be passed to the classifier. Remember that we want to **use the hidden state of the last sequence element.**
- Feed the extracted output into the classifier.

Hint: Keep in mind that `Conv1d` and `RNN` layers expect inputs in different dimension orders. Therefore, when connecting these layers, make sure to reorder (`permute`) the tensor appropriately.

3. LSTM: feature extractor → LSTM layer → Final hidden state → Classifier

After the **feature extractor**, you will use a **Long Short-Term Memory layer** to model the **temporal dependencies** over the **compressed time features** produced by the feature extractor network.

- Use the PyTorch module `torch.nn.LSTM` to implement this layer.
- The LSTM should consist of a **single layer** with 128 hidden units
- Be careful about setting `batch_first` parameter when creating the LSTM. **The shape of the input to the LSTM must align with this parameter.**
- After the LSTM processes the sequence in `forward`, you must correctly determine the output that will be passed to the classifier. Remember that we want to **use the hidden state of the last sequence element.**
- Feed the extracted output into the classifier.

Hint 1: Be aware that the interfaces of `torch.nn.RNN` and `torch.nn.LSTM` differ in what the `forward` function outputs. Make sure that you adapt your handling of the output accordingly.

Hint 2: Keep in mind that `Conv1d` and `LSTM` layers expect inputs in different dimension orders. Therefore, when connecting these layers, make sure to reorder (`permute`) the tensor appropriately.

4. Attention + Class Token: feature extractor → Add Class Token → Self-Attention → Extract Class Token → Classifier

After passing the input through the **feature extractor**, the compressed sequence features will be further processed by a **self-attention mechanism**. To help the network better summarize the entire sequence information and their temporal relations into a **single representation**, you will introduce a `class token`.

What is a Class Token? A `class token` is a **learnable vector** that is prepended (or appended) to the sequence of features before feeding into the attention module. During training, the model learns to use this token to gather global information from the entire sequence through the attention mechanism. This idea is inspired by techniques used in transformer architectures like ViT (Vision Transformer) and has proven effective for tasks that require summarizing a sequence into a fixed-size output.

- Create a `class_token` parameter using `torch.nn.Parameter` and initialize it with a normal distribution $\mathcal{N}(0, 1)$ within **correct shape** inside `__init__` method.
- Define `self_attention` layer using PyTorch's `torch.nn.MultiheadAttention` module inside `__init__` method, with **one attention head** and an **embedding dimension of 128**. *This layer internally constructs the query $\mathbf{W}^{(Q)}$, key $\mathbf{W}^{(K)}$ and value matrices $\mathbf{W}^{(V)}$ used during attention computation.*
- Be careful about setting `batch_first` parameter when defining the `self_attention`. **The shape of the input to the attention module must align with this parameter.**
- In `forward`, concatenate the class token to the beginning or end of the feature sequence after the feature extractor. Make sure that the same class token is replicated for all batch elements.
- Apply `self_attention` over the extended sequence (including the class token).
- After the attention operation, extract the transformed `class_token` (based on where you inserted it).
- Feed the extracted class token into the classifier.

Hint 1: You are expected to use the final class token after attention, which represents the entire input sequence.

Hint 2: Keep in mind that `Conv1d` and `MultiheadAttention` layers expect inputs in different dimension orders. Therefore, when connecting these layers, make sure to reorder (`permute`) the tensor appropriately.

To keep the model implementation clean and organized, you are encouraged to use Python's **match-case statement** (available in Python 3.10 and above) to handle the different model behaviors in the `forward` function. This avoids long `if ... elif ...` chains and makes the architecture selection more readable and efficient.

Advice: This is surely the most difficult part of this assignment. Maybe it would be advisable to implement and test this piece by piece, starting with the most simple network (Global Pooling), then implement the training code in Task 2.8, and run the training in Task 2.9. Only after you have achieved reasonable results, attempt to implement the RNN and LSTM networks, and run their training in Task 2.9. Finally, implement the attention-based part of the network, which requires more effort and debugging. **Good luck!**

```
class SleepStageClassifier(torch.nn.Module):
    """
    Sleep stage classification model with optional temporal module.

    Parameters
    -----
    model_type : str
        Type of sequence model to use:
        - "pool" : global average pooling
        - "rnn" : RNN
        - "lstm" : LSTM
        - "attn" : self-attention with CLS token

    hidden_size : int, default=128
        Feature dimension from CNN feature extractor.

    O : int
        Number of output classes.
    """
    def __init__(self, model_type, hidden_size = 128, O = 0):
        super().__init__()

        # Define the extractor and model type
        self.feature_extractor = feature_extractor(output_channels=hidden_size)
        self.model_type = model_type

        # Define the classifier that will be used for all model variants
        self.classifier = torch.nn.Sequential(
            torch.nn.Linear(hidden_size, 64),
            torch.nn.GELU(),
            torch.nn.Dropout(0.3),
            torch.nn.Linear(64, O)
        )

        # Define optional layers based on model type
        match model_type:
            case "pool":
                pass

            case "rnn":
                self.rnn = torch.nn.RNN(input_size=hidden_size, hidden_size=hidden_size, num_layers=1, batch_first=True)

            case "lstm":
                self.lstm = torch.nn.LSTM(input_size=hidden_size, hidden_size=hidden_size, num_layers=1, batch_first=True)

            case "attn":
                self.attn = torch.nn.MultiheadAttention(embed_dim=hidden_size, num_heads=1, batch_first=True)
                self.class_token = torch.nn.Parameter(torch.randn(1, 1, hidden_size))

    def forward(self, x):
        """Extract class logits from the data

        Parameters
        -----
        x: torch.tensor in shape B x 1 x 3000

        Returns
        -----
        class logits, a torch.tensor in shape B x O
        """

        # x is (B, 1, 3000). Feature extractor expects (B, C, S)
        x = self.feature_extractor(x)

        # After CNN, shape is (B, hidden_size, L_reduced).
        # Temporal modules expect (B, S, Features), so we permute.
        x = x.permute(0, 2, 1)

        match self.model_type:
            case "pool":
                x = torch.mean(x, dim=1)

            case "rnn":
                _, h_n = self.rnn(x)
                x = h_n.squeeze(0)
```



```

        case "lstm":
            _, (h_n, _) = self.lstm(x)
            x = h_n.squeeze(0)

        case "attn":
            cls_token = self.class_token.expand(x.shape[0], -1, -1)
            x = torch.cat((cls_token, x), dim=1)
            x, _ = self.attn(x, x, x)
            x = x[:, 0, :]

    x = self.classifier(x)

    return x

```

✓ Network Training

In this section, you will implement a training loop function that handles the model optimization process. Using this function, you will train all four model variants (Global Pooling, RNN, LSTM, Attention) on the prepared EEG dataset.

✓ Task 2.8: Training Loop

Implement a **training loop function** to optimize your sleep stage classification models. The goal is to train all four model variants on the prepared dataset and compare their performance under different loss-weighting strategies.

Begin by training each model using **the standard categorical cross-entropy loss** to establish a baseline and compare the performance of the four models to understand how well each architecture handles the task. After that, apply two methods: **Class-Weighted Cross Entropy** and **DynaMit (Dynamic Bias Mitigation)** to help **mitigate the class imbalance problem** commonly found in sleep stage data. Therefore, the function should support these three methods for use in the upcoming training tasks, based on the input parameter `weighting_strategy`, see below. Specifically, you will experiment with the following three approaches:

1. Standard Cross Entropy Loss

When `weighting_strategy = "none"`, standard cross-entropy loss should be used.

2. Class-Weighted Cross Entropy

When `weighting_strategy = "global"`, global weights shall be computed. This can be achieved by calling the `class_weights` function defined in Task 2.3, using the appropriate targets from the dataset.

3. DynaMit

When `weighting_strategy = "dynamit"`, per-batch weights shall be computed. This can be achieved by calling the `class_weights` function defined in Task 2.3, using the targets for the current batch.

Hint: You can use the functional variation of cross-entropy loss, which is defined as `torch.nn.functional.cross_entropy`, where you can pass the logits, the targets and the class weights. Note that this requires the class weights to be on the same device as the logits and the targets.

In summary, here is a clean and concise description of what the training function should do:

- Support three loss strategies
- Use the **SGD optimizer** with $\eta = 0.01$ and momentum of 0.9 for model parameter updates.
- Perform a training loop over the training dataset.
- Run a validation loop on the validation dataset after each epoch.
- Track and store the **average training loss** and **validation accuracy** after each epoch for monitoring convergence.
- Return the **training losses** and **validation accuracies**.

```

def train(model, weighting_strategy="none", epochs=30):
    """
    Train a sleep stage classification model.

    Parameters
    -----
    model : torch.nn.Module
        Sleep stage classifier.

    weighting_strategy : str
        Class weighting strategy:
        - "none"      : no class weights
        - "global"    : dataset-level class weights
        - "dynamit"   : batch-level (dynamic) class weights
    """

```

```

epochs : int, default=30
    Number of training epochs.

Returns
-----
train_losses : list of float
    Average training loss per epoch.

valid_accuracies : list of float
    Validation accuracy per epoch.
"""

if weighting_strategy not in ("none", "global", "dynamit"):
    raise ValueError(f"The given weighting strategy '{weighting_strategy}' is not known")

# Send the model to device
model = model.to(device)

# Define optimizer
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# Precompute global weights if needed
global_weights = None
if weighting_strategy == "global":
    global_weights = class_weights(T_train).to(device)

# Store average training loss and validation accuracy
train_losses, valid_accuracies = [], []

for epoch in range(epochs):
    # Training mode
    model.train()
    total_loss = 0

    for x, t in train_dataloader:
        # Determine class weights based on strategy
        current_weights = None
        if weighting_strategy == "dynamit":
            current_weights = class_weights(t).to(device)
        elif weighting_strategy == "global":
            current_weights = global_weights

        # Move data to device
        x, t = x.to(device), t.to(device)

        # Forward pass
        optimizer.zero_grad()
        logits = model(x)

        # Compute loss
        J = torch.nn.functional.cross_entropy(logits, t, weight=current_weights)

        # Backward pass and optimization
        J.backward()
        optimizer.step()

        total_loss += J.item() * x.size(0)

    # Compute average training loss
    avg_train_loss = total_loss / len(train_dataloader.dataset)
    train_losses.append(avg_train_loss)

    # Validation phase
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for x_val, t_val in valid_dataloader:
            x_val, t_val = x_val.to(device), t_val.to(device)
            logits_val = model(x_val)
            preds = torch.argmax(logits_val, dim=1)
            correct += (preds == t_val).sum().item()
            total += t_val.size(0)

    valid_accuracy = correct / total
    valid_accuracies.append(valid_accuracy)

    print(f"Epoch [{epoch+1}/{epochs}] -- Train Loss: {avg_train_loss:.4f} -- Val Acc: {valid_accuracy*100:.2f}%")

return train_losses, valid_accuracies

```

Task 2.9: Plot Training Loss and Validation Accuracy

Implement a function that plots the training loss and validation accuracy over epochs to visually assess the model's learning progress. These plots help determine whether the model is converging, overfitting, or underfitting during training.

Smoothly decreasing training loss and increasing validation accuracy typically indicate good convergence.

```
def plot(train_losses, valid_accuracies):
    """
    Plot training loss and validation accuracy over epochs.

    Parameters
    -----
    train_losses : list of float
        Training loss per epoch.

    valid_accuracies : list of float
        Validation accuracy per epoch.
    """
    # --- Plot Training Loss & Validation Accuracy ---

    epochs = range(1, len(train_losses) + 1)
    fig, ax = pyplot.subplots(figsize=(8, 5))

    # Training loss (Left Y-axis)
    ax.plot(epochs, train_losses, label="Training Loss", color='b', marker='o')
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss", color='b')
    ax.tick_params(axis='y', labelcolor='b')

    # Validation Accuracy (Right Y-axis)
    ax2 = ax.twinx()
    # Convert accuracy to percentage
    valid_acc_perc = [acc * 100 for acc in valid_accuracies]
    ax2.plot(epochs, valid_acc_perc, label="Validation Accuracy", color='g', linestyle="dashed")
    ax2.set_ylabel("Accuracy (%)", color='g')
    ax2.tick_params(axis='y', labelcolor='g')

    # Titles and Legends
    pyplot.title("Training Loss vs Validation Accuracy")
    ax.legend(loc="upper left")
    ax2.legend(loc="lower left")

    pyplot.grid(True, linestyle="--", alpha=0.6)
    pyplot.show()
```

Task 2.10: Train Four Model Variants with Standard Categorical Cross Entropy

Separately train **all four model variants** using the **standard categorical cross-entropy loss** to compare different model topologies for sleep stage classification. Each model uses the same feature extractor topology and classifier head, but differs in **how it models temporal dependencies** (e.g., RNN, LSTM, Attention).

Your goal is to evaluate **how well each architecture performs without any special handling for class imbalance**.

Follow these steps for each model variant:

- Initialize the model variant with the appropriate `model_type` (using network class previously implemented in Task 2.7).
- Run the training loop (using training function previously implemented in Task 2.8) for **30 epochs** (or more).
- Plot the training results to monitor learning progress and performance.

Once trained, you will be able to compare the performance of all four model variants under the same training conditions.

WARNING: The RNN model might not perform well. Maybe work on the Global Pooling model first, and then directly transition to the LSTM. Keep the RNN for later.

```
# ----- Pooled model Training -----

# Initialize the model
pooled_model = SleepStageClassifier(model_type="pool")

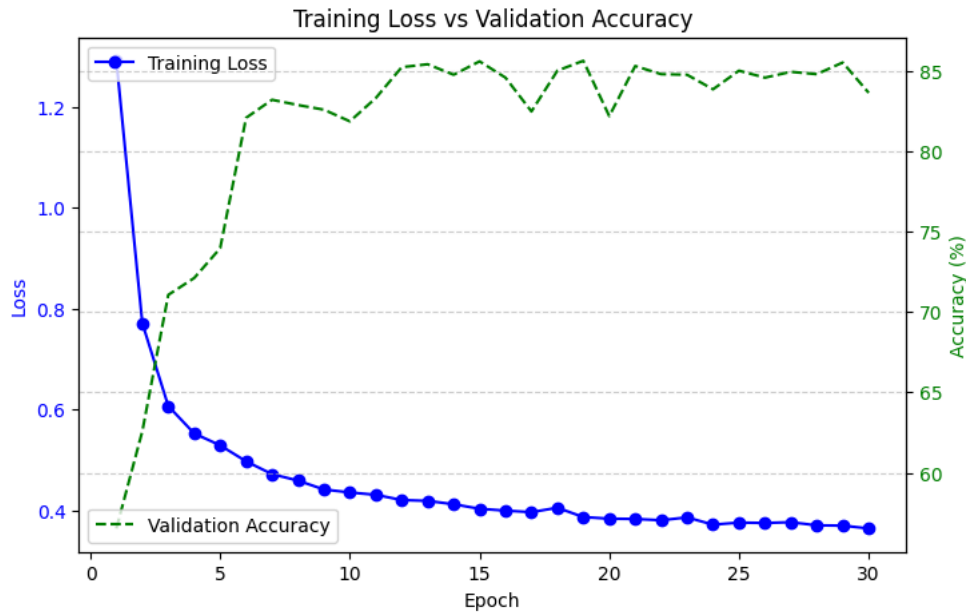
# Run the training
pooled_train_losses, pooled_valid_accuracies = train(pooled_model, weighting_strategy="none", epochs=30)

# Plot training results
plot(pooled_train_losses, pooled_valid_accuracies)
```

```

Epoch [1/30] -- Train Loss: 1.2914 -- Val Acc: 56.54%
Epoch [2/30] -- Train Loss: 0.7711 -- Val Acc: 62.59%
Epoch [3/30] -- Train Loss: 0.6070 -- Val Acc: 71.06%
Epoch [4/30] -- Train Loss: 0.5525 -- Val Acc: 72.11%
Epoch [5/30] -- Train Loss: 0.5290 -- Val Acc: 73.96%
Epoch [6/30] -- Train Loss: 0.4973 -- Val Acc: 82.07%
Epoch [7/30] -- Train Loss: 0.4719 -- Val Acc: 83.19%
Epoch [8/30] -- Train Loss: 0.4591 -- Val Acc: 82.87%
Epoch [9/30] -- Train Loss: 0.4414 -- Val Acc: 82.58%
Epoch [10/30] -- Train Loss: 0.4356 -- Val Acc: 81.85%
Epoch [11/30] -- Train Loss: 0.4309 -- Val Acc: 83.30%
Epoch [12/30] -- Train Loss: 0.4205 -- Val Acc: 85.22%
Epoch [13/30] -- Train Loss: 0.4189 -- Val Acc: 85.40%
Epoch [14/30] -- Train Loss: 0.4122 -- Val Acc: 84.75%
Epoch [15/30] -- Train Loss: 0.4032 -- Val Acc: 85.58%
Epoch [16/30] -- Train Loss: 0.3995 -- Val Acc: 84.57%
Epoch [17/30] -- Train Loss: 0.3965 -- Val Acc: 82.47%
Epoch [18/30] -- Train Loss: 0.4053 -- Val Acc: 85.01%
Epoch [19/30] -- Train Loss: 0.3868 -- Val Acc: 85.62%
Epoch [20/30] -- Train Loss: 0.3836 -- Val Acc: 82.18%
Epoch [21/30] -- Train Loss: 0.3830 -- Val Acc: 85.30%
Epoch [22/30] -- Train Loss: 0.3804 -- Val Acc: 84.79%
Epoch [23/30] -- Train Loss: 0.3862 -- Val Acc: 84.75%
Epoch [24/30] -- Train Loss: 0.3717 -- Val Acc: 83.85%
Epoch [25/30] -- Train Loss: 0.3759 -- Val Acc: 85.01%
Epoch [26/30] -- Train Loss: 0.3750 -- Val Acc: 84.57%
Epoch [27/30] -- Train Loss: 0.3766 -- Val Acc: 84.93%
Epoch [28/30] -- Train Loss: 0.3703 -- Val Acc: 84.79%
Epoch [29/30] -- Train Loss: 0.3698 -- Val Acc: 85.51%
Epoch [30/30] -- Train Loss: 0.3642 -- Val Acc: 83.63%

```



```

# ----- RNN Training -----

# Initialize the model
rnn_model = SleepStageClassifier(model_type="rnn")

# Run the training
rnn_train_losses, rnn_valid_accuracies = train(rnn_model, weighting_strategy="none", epochs=30)

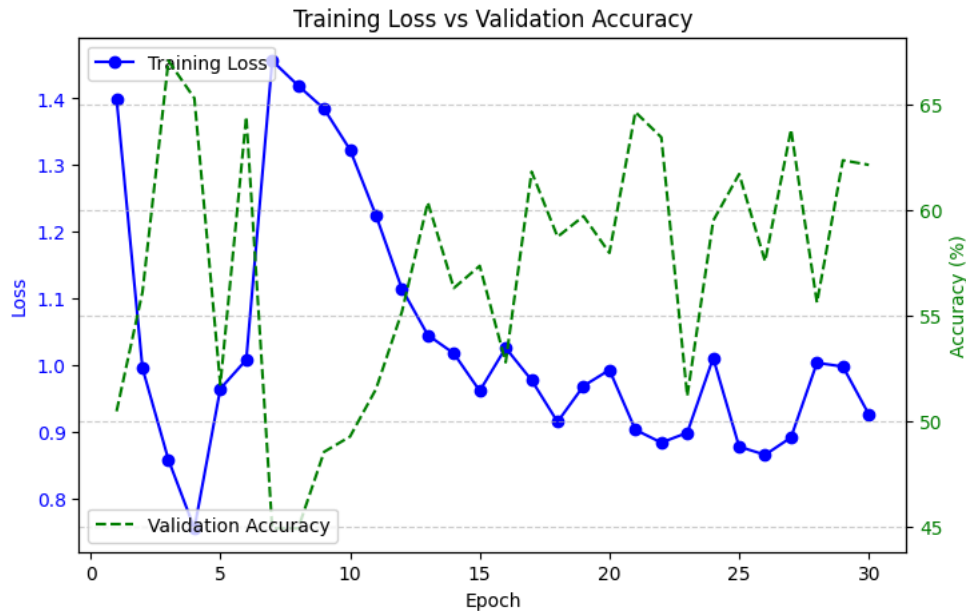
# Plot training results
plot(rnn_train_losses, rnn_valid_accuracies)

```

```

Epoch [1/30] -- Train Loss: 1.3994 -- Val Acc: 50.45%
Epoch [2/30] -- Train Loss: 0.9953 -- Val Acc: 56.10%
Epoch [3/30] -- Train Loss: 0.8581 -- Val Acc: 67.04%
Epoch [4/30] -- Train Loss: 0.7550 -- Val Acc: 65.27%
Epoch [5/30] -- Train Loss: 0.9646 -- Val Acc: 51.72%
Epoch [6/30] -- Train Loss: 1.0080 -- Val Acc: 64.40%
Epoch [7/30] -- Train Loss: 1.4565 -- Val Acc: 44.91%
Epoch [8/30] -- Train Loss: 1.4193 -- Val Acc: 44.91%
Epoch [9/30] -- Train Loss: 1.3845 -- Val Acc: 48.53%
Epoch [10/30] -- Train Loss: 1.3231 -- Val Acc: 49.26%
Epoch [11/30] -- Train Loss: 1.2247 -- Val Acc: 51.50%
Epoch [12/30] -- Train Loss: 1.1145 -- Val Acc: 55.16%
Epoch [13/30] -- Train Loss: 1.0451 -- Val Acc: 60.34%
Epoch [14/30] -- Train Loss: 1.0176 -- Val Acc: 56.28%
Epoch [15/30] -- Train Loss: 0.9614 -- Val Acc: 57.33%
Epoch [16/30] -- Train Loss: 1.0258 -- Val Acc: 52.77%
Epoch [17/30] -- Train Loss: 0.9774 -- Val Acc: 61.79%
Epoch [18/30] -- Train Loss: 0.9149 -- Val Acc: 58.71%
Epoch [19/30] -- Train Loss: 0.9688 -- Val Acc: 59.69%
Epoch [20/30] -- Train Loss: 0.9928 -- Val Acc: 57.95%
Epoch [21/30] -- Train Loss: 0.9030 -- Val Acc: 64.61%
Epoch [22/30] -- Train Loss: 0.8837 -- Val Acc: 63.42%
Epoch [23/30] -- Train Loss: 0.8986 -- Val Acc: 51.18%
Epoch [24/30] -- Train Loss: 1.0106 -- Val Acc: 59.47%
Epoch [25/30] -- Train Loss: 0.8777 -- Val Acc: 61.68%
Epoch [26/30] -- Train Loss: 0.8654 -- Val Acc: 57.55%
Epoch [27/30] -- Train Loss: 0.8915 -- Val Acc: 63.78%
Epoch [28/30] -- Train Loss: 1.0035 -- Val Acc: 55.60%
Epoch [29/30] -- Train Loss: 0.9978 -- Val Acc: 62.33%
Epoch [30/30] -- Train Loss: 0.9262 -- Val Acc: 62.12%

```



```

# ----- LSTM Training -----

# Initialize the model
lstm_model = SleepStageClassifier(model_type="lstm")

# Run the training
lstm_train_losses, lstm_valid_accuracies = train(lstm_model, weighting_strategy="none", epochs=30)

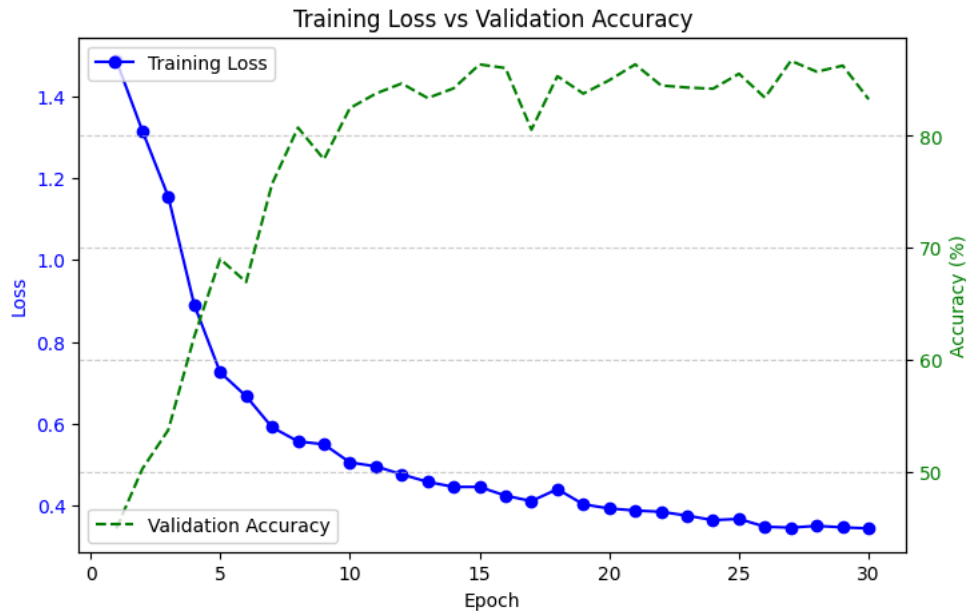
# Plot training results
plot(lstm_train_losses, lstm_valid_accuracies)

```

```

Epoch [1/30] -- Train Loss: 1.4866 -- Val Acc: 44.91%
Epoch [2/30] -- Train Loss: 1.3156 -- Val Acc: 50.24%
Epoch [3/30] -- Train Loss: 1.1548 -- Val Acc: 53.71%
Epoch [4/30] -- Train Loss: 0.8907 -- Val Acc: 62.04%
Epoch [5/30] -- Train Loss: 0.7257 -- Val Acc: 69.00%
Epoch [6/30] -- Train Loss: 0.6675 -- Val Acc: 66.93%
Epoch [7/30] -- Train Loss: 0.5916 -- Val Acc: 75.70%
Epoch [8/30] -- Train Loss: 0.5571 -- Val Acc: 80.73%
Epoch [9/30] -- Train Loss: 0.5499 -- Val Acc: 77.91%
Epoch [10/30] -- Train Loss: 0.5058 -- Val Acc: 82.47%
Epoch [11/30] -- Train Loss: 0.4961 -- Val Acc: 83.77%
Epoch [12/30] -- Train Loss: 0.4765 -- Val Acc: 84.68%
Epoch [13/30] -- Train Loss: 0.4581 -- Val Acc: 83.38%
Epoch [14/30] -- Train Loss: 0.4459 -- Val Acc: 84.24%
Epoch [15/30] -- Train Loss: 0.4458 -- Val Acc: 86.38%
Epoch [16/30] -- Train Loss: 0.4245 -- Val Acc: 86.06%
Epoch [17/30] -- Train Loss: 0.4112 -- Val Acc: 80.55%
Epoch [18/30] -- Train Loss: 0.4404 -- Val Acc: 85.33%
Epoch [19/30] -- Train Loss: 0.4033 -- Val Acc: 83.77%
Epoch [20/30] -- Train Loss: 0.3932 -- Val Acc: 84.97%
Epoch [21/30] -- Train Loss: 0.3883 -- Val Acc: 86.38%
Epoch [22/30] -- Train Loss: 0.3851 -- Val Acc: 84.50%
Epoch [23/30] -- Train Loss: 0.3758 -- Val Acc: 84.32%
Epoch [24/30] -- Train Loss: 0.3646 -- Val Acc: 84.21%
Epoch [25/30] -- Train Loss: 0.3678 -- Val Acc: 85.55%
Epoch [26/30] -- Train Loss: 0.3487 -- Val Acc: 83.41%
Epoch [27/30] -- Train Loss: 0.3464 -- Val Acc: 86.71%
Epoch [28/30] -- Train Loss: 0.3508 -- Val Acc: 85.73%
Epoch [29/30] -- Train Loss: 0.3470 -- Val Acc: 86.27%
Epoch [30/30] -- Train Loss: 0.3445 -- Val Acc: 83.27%

```



```

# ----- Attention Training -----

# Initialize the model
attn_model = SleepStageClassifier(model_type="attn")

# Run the training
attn_train_losses, attn_valid_accuracies = train(attn_model, weighting_strategy="none", epochs=30)

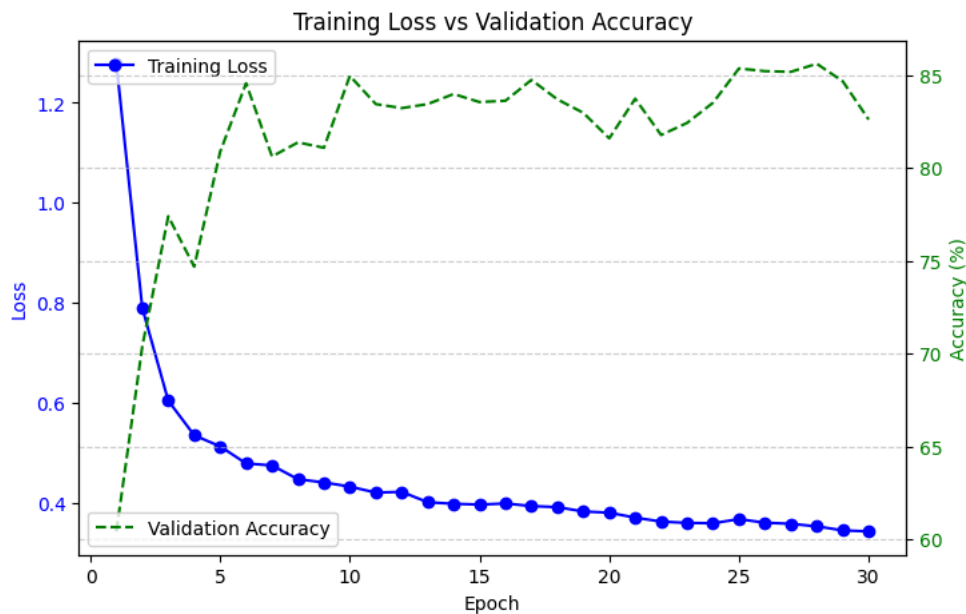
# Plot training results
plot(attn_train_losses, attn_valid_accuracies)

```

```

Epoch [1/30] -- Train Loss: 1.2781 -- Val Acc: 60.41%
Epoch [2/30] -- Train Loss: 0.7892 -- Val Acc: 70.41%
Epoch [3/30] -- Train Loss: 0.6042 -- Val Acc: 77.40%
Epoch [4/30] -- Train Loss: 0.5350 -- Val Acc: 74.68%
Epoch [5/30] -- Train Loss: 0.5128 -- Val Acc: 80.88%
Epoch [6/30] -- Train Loss: 0.4787 -- Val Acc: 84.57%
Epoch [7/30] -- Train Loss: 0.4747 -- Val Acc: 80.62%
Epoch [8/30] -- Train Loss: 0.4471 -- Val Acc: 81.38%
Epoch [9/30] -- Train Loss: 0.4407 -- Val Acc: 81.09%
Epoch [10/30] -- Train Loss: 0.4323 -- Val Acc: 84.97%
Epoch [11/30] -- Train Loss: 0.4203 -- Val Acc: 83.45%
Epoch [12/30] -- Train Loss: 0.4219 -- Val Acc: 83.23%
Epoch [13/30] -- Train Loss: 0.4011 -- Val Acc: 83.45%
Epoch [14/30] -- Train Loss: 0.3980 -- Val Acc: 83.99%
Epoch [15/30] -- Train Loss: 0.3964 -- Val Acc: 83.56%
Epoch [16/30] -- Train Loss: 0.3986 -- Val Acc: 83.63%
Epoch [17/30] -- Train Loss: 0.3934 -- Val Acc: 84.75%
Epoch [18/30] -- Train Loss: 0.3913 -- Val Acc: 83.70%
Epoch [19/30] -- Train Loss: 0.3826 -- Val Acc: 82.98%
Epoch [20/30] -- Train Loss: 0.3802 -- Val Acc: 81.60%
Epoch [21/30] -- Train Loss: 0.3703 -- Val Acc: 83.74%
Epoch [22/30] -- Train Loss: 0.3625 -- Val Acc: 81.78%
Epoch [23/30] -- Train Loss: 0.3597 -- Val Acc: 82.43%
Epoch [24/30] -- Train Loss: 0.3591 -- Val Acc: 83.52%
Epoch [25/30] -- Train Loss: 0.3673 -- Val Acc: 85.37%
Epoch [26/30] -- Train Loss: 0.3602 -- Val Acc: 85.22%
Epoch [27/30] -- Train Loss: 0.3580 -- Val Acc: 85.19%
Epoch [28/30] -- Train Loss: 0.3528 -- Val Acc: 85.62%
Epoch [29/30] -- Train Loss: 0.3448 -- Val Acc: 84.68%
Epoch [30/30] -- Train Loss: 0.3427 -- Val Acc: 82.61%

```



Evaluation and Bias Mitigation

In this section, you will evaluate the performance of all four trained model variants on the test set using three key metrics:

- **Accuracy:** the proportion of correctly predicted samples across the entire test set.
- **Balanced Accuracy:** the average of recall scores across all classes, helping to account for class imbalance.
- **Per-Class Accuracy:** individual accuracy for each sleep stage, providing insights into which stages are predicted well and which are not.

Based on these metrics, you will select the best-performing model — the one that achieves the most reliable results.

Once the best model is identified, you will retrain it using two different strategies to mitigate the class imbalance issue:

1. Class-Weighted Cross Entropy

Incorporates static global class weights into the loss function based on training set distribution.

2. DynaMit

Applies dynamic per-batch class weights during training, adapting to local imbalances.

This comparison allows you to assess the effectiveness of bias mitigation strategies in improving fairness and performance, especially on underrepresented sleep stages.

Task 2.11: Evaluation Function

Write a function to evaluate a **trained model** on the **test set**.

The goal is to assess the model's performance using the following metrics: **Overall Accuracy**, **Balanced Accuracy** and **Per-Class Accuracy**.

Per-Class Accuracy

$$\forall \tau \in \{1, \dots, O\} : \text{Per-Class Accuracy}_\tau = \frac{TP_\tau}{N_\tau}$$

Balanced Accuracy:

$$\text{Balanced Accuracy} = \frac{1}{O} \sum_{\tau=1}^O \text{Per-Class Accuracy}_\tau$$

where TP_τ is the number of correctly classified samples in class τ . Your evaluation function should:

- Run the test loop
- Use `sklearn.metrics` to compute:
 - `accuracy_score` (for overall accuracy)
 - `balanced_accuracy_score` (for balanced accuracy)
 - compute the diagonal of the normalized `confusion_matrix` (for per-class accuracy); this is a vector with O elements
- Return all metrics

```
from sklearn.metrics import accuracy_score, balanced_accuracy_score, confusion_matrix

def evaluate(model):
    """
    Evaluate a trained sleep stage classification model.

    Parameters
    -----
    model : torch.nn.Module
        Trained model to evaluate.

    Returns
    -----
    overall_acc : float
        Overall classification accuracy.

    balanced_acc : float
        Balanced accuracy across classes.

    per_class_acc : numpy.ndarray
        Per-class accuracy from normalized confusion matrix.
    """
    # Define empty lists to store predictions and their corresponding targets
    all_preds, all_targets = [], []

    # Test loop
    model.eval()
    with torch.no_grad():
        for x_test, t_test in test_dataloader:
            x_test = x_test.to(device)
            logits = model(x_test)
            preds = torch.argmax(logits, dim=1)

            all_preds.append(preds.cpu().numpy())
            all_targets.append(t_test.numpy())

    # Concatenate all batches
    all_preds = numpy.concatenate(all_preds)
    all_targets = numpy.concatenate(all_targets)

    # Compute evaluation metrics via sklearn
    overall_acc = accuracy_score(all_targets, all_preds)
    balanced_acc = balanced_accuracy_score(all_targets, all_preds)

    # Compute per class accuracy via confusion matrix
    # 'true' normalization ensures we get recall/per-class accuracy on the diagonal
    cm = confusion_matrix(all_targets, all_preds, normalize='true')
    per_class_acc = cm.diagonal()

    return overall_acc, balanced_acc, per_class_acc
```


Implement a function that visualizes the evaluation results of all four model variants to better compare their performance.

```
def plot_evaluation(all_model_results):
    """
    Plot per-class evaluation results for multiple models.

    Parameters
    -----
    all_model_results : dict
        Dictionary where keys are model names and values are tuples:
        (overall_acc, balanced_acc, per_class_acc)
    """

    num_models = len(all_model_results)
    rows = (num_models + 1) // 2
    fig, axs = pyplot.subplots(rows, 2, figsize=(14, 5 * rows))
    axs = axs.flatten()

    for i, (model_name, (overall_acc, balanced_acc, per_class_acc)) in enumerate(all_model_results.items()):
        ax = axs[i]

        # Plot each per-class accuracy with bar plot
        bars = ax.bar(stage_names.values(), per_class_acc, color=[stage_colors[i] for i in range(5)])

        # Annotate the value of per-class-acc to the top of the bars
        for bar, acc in zip(bars, per_class_acc):
            ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f"{acc:.2f}", ha='center', fontsize=10)

        ax.set_ylim(0, 1.05)
        ax.set_ylabel("Accuracy")
        ax.set_title(model_name, fontsize=12, pad=30)
        ax.grid(axis='y', linestyle='--', alpha=0.5)

        # Accuracy box just below title
        acc_text = f"Overall Acc: {overall_acc * 100:.2f}%   Balanced Acc: {balanced_acc * 100:.2f}%"
        ax.text(
            0.5, 1.07, acc_text, transform=ax.transAxes,
            ha="center", va="top",
            bbox=dict(facecolor='lightgreen', edgecolor='gray', boxstyle='round,pad=0.3'),
            fontsize=10
        )

    # Remove any unused subplots
    for j in range(i + 1, len(axs)):
        fig.delaxes(axs[j])

    pyplot.tight_layout()
    pyplot.show()
```

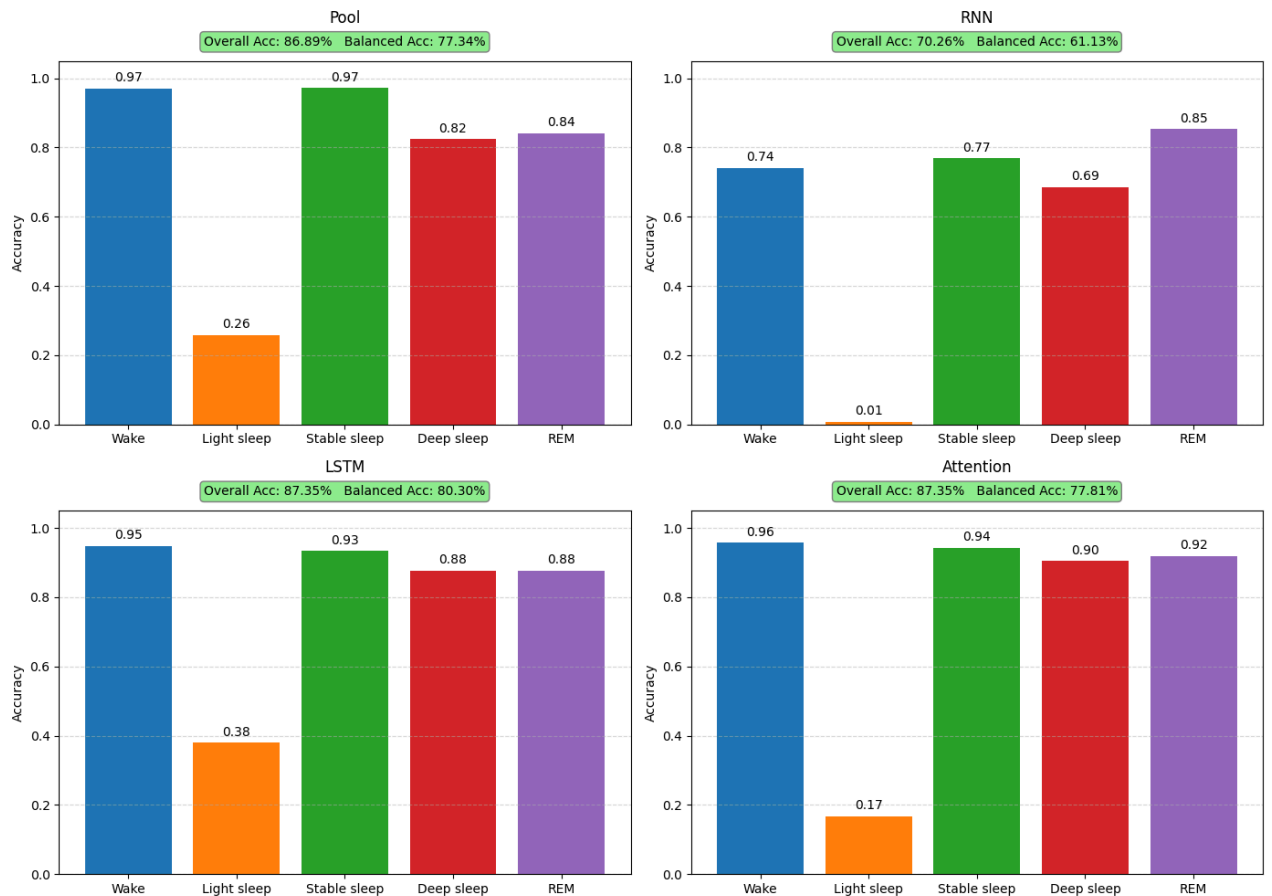
▼ Task 2.13: Plot All Evaluation Metrics for All Models

Run the evaluation on test set for each model and visualize the evaluation results of all four model variants to better compare their performance.

These plots will help you compare the strengths and weaknesses of each architecture and identify which model performs best across both highly and sparsely represented sleep stages.

```
# run evaluation for all models
all_model_results = {
    "Pool": evaluate(pooled_model),
    "RNN": evaluate(rnn_model),
    "LSTM": evaluate(lstm_model),
    "Attention": evaluate(attn_model)
}

# Compare results
plot_evaluation(all_model_results)
```



Task 2.14: Bias Mitigation

In this task, you will address the issue of **class imbalance** by retraining the best-performing model as determined in the previous task using two different bias mitigation strategies.

Class imbalance is a common challenge in sleep stage classification, where some stages (like Light Sleep) may be underrepresented. This task focuses on evaluating whether loss function modifications can improve fairness and accuracy across all classes.

Steps to Follow:

- Select the best model variant based on highest balanced accuracy
- Initialize the same model architecture twice
- Train each one of these models using:
 1. **global** weighting strategy
 2. batch-local **DynaMit** sampling
- Evaluate the retrained models on the test set
- Plot and compare the metrics from all three runs (Standard Cross Entropy, Weighted CCE, DynaMit) of the same model.

Analyze how each mitigation strategy affects performance across different sleep stages.

```
# Based on the results in all_model_results, LSTM had the best balanced accuracy (~0.80)
BEST_MODEL_TYPE = "lstm"

# ---- Training with Class Weighted CCE ----

# Initialize the best model topology
best_model_global_weighted = SleepStageClassifier(model_type=BEST_MODEL_TYPE)

# Train the best_model with fixed global class weights
global_train_losses, global_valid_accs = train(best_model_global_weighted, weighting_strategy="global", epochs=30)

# ---- Training with DynaMit ----

# Initialize the best model topology again
```

```
best_model_dynamit = SleepStageClassifier(model_type=BEST_MODEL_TYPE)

# Train the best_model with dynamic weights
dynamit_train_losses, dynamit_valid_accs = train(best_model_dynamit, weighting_strategy="dynamit", epochs=30)

# run evaluation for all three models
best_model_results = {
    "LSTM (Standard CCE)": all_model_results["LSTM"],
    "LSTM (Global Weighting)": evaluate(best_model_global_weighted),
    "LSTM (DynaMit)": evaluate(best_model_dynamit)
}
```